

Assignment 01 (Resit)

Design and Analysis of a Polynomial Class

Khiem Nguyen

1 Foreword: Story behind this Resit Assignment

In the main semester, unfortunately several students took the easy way out: copy-pasting solutions directly from platforms like ChatGPT, Claude, or Gemini. They obtained correct outputs in less than an hour, but without actual understanding and skills of writing code on their own. When faced with manual coding assessments or follow-up questions, the lack of actual understanding became immediately apparent. The evidence was that 23/92 students failed in the main exam and scored almost zero scores on questions testing C++ theory and code writing skills. However, it must be emphasized that the exam is highly achievable because 13 students with actual learning could obtain A1/A2 grades (8 students with A1 and 5 students with A2.)

Using GenAI to write your code is not learning; it is outsourcing your education.

Since you are taking this resit due to a breach of the academic honor code, this assignment is explicitly designed to change how you interact with GenAI. Instead of pretending you do not use GenAI, **you are required to use it**. However, you will use it as a baseline reference. Your task is to learn from it, critique it, and if possible, improve it, and finally write your own robust C++ implementation from scratch.

Remember: **You will have to write C++ code on paper in your final paper exam.** Copying without understanding will guarantee another failure. Use this assignment to actually build your skills.

2 Assignment Summary: The C++ Polynomial Class

Your task is to design and implement a C++ class named `Polynomial` that mimics the primary behavior of Python's `numpy.poly1d`.

To preserve the simplicity and safety of modern C++, you must store the polynomial's coefficients using standard containers and modern initializer syntax.

3 Core specifications

- Storage representation:** Store your coefficients in a `std::vector<double>`. Unlike NumPy, which stores coefficients in descending order of power, you may choose to store them in ascending order (i.e., `coefficients[i]` corresponds to the term x^i) to make mathematical indexing easier. Clearly document your choice.
- Constructors**
 - A default constructor initializing the zero polynomial.
 - A constructor that accepts a `std::initializer_list<double>` to enable intuitive initializations like `Polynomial p {1.0, 2.0, 3.0};`.

3. **Algebraic Operations (Operator Overloading):** Implement the following operators to allow natural algebraic expressions :

- Addition (+): Combines two polynomials.
- Subtraction (-): Subtracts one polynomial from another.
- Multiplication (*): Multiplies two polynomials (hint: convolution of coefficient vectors) and also multiplies a polynomial by a scalar.
- Evaluation (Function Call `operator()`): Allows evaluating the polynomial at a value x via `p(x)` (implement using Horner's Method for numerical efficiency).

4. I/O Streaming (`operator<<`):

Overload the insertion operator so that printing `p` to `std::cout` produces a clean, readable string representation (e.g., $3x^2 + 2x + 1$). Clean up edge cases like zero coefficients, negative coefficients, and x^1 or x^0 formatting. That is, you should output `42x` instead of `42x^1` and `42` instead of `42x^0`.

4 Your Learning & Comparative Analysis Tasks

This assignment is divided into three consecutive milestones. You must document your steps for each milestone in your report.

Step 1: Establish the GenAI Baseline

1. Prompt a GenAI tool (such as ChatGPT, Claude, or Gemini) to generate a C++ Polynomial class meeting the specifications described above.
2. Copy the generated code into a file name `genai_baseline.cpp`
3. Try to compile and run it. **Do not fix or improve anything that do not meet the specifications. You only need to fix the code until it can compile and run it.** In other words, try to keep the code generated by AI as original as possible.

Step 2: Write Your Own Implementation

1. Write your own clean, distinct implementation from scratch in `Polynomial.hpp` and `Polynomial.cpp`.
2. Do not copy and paste the GenAI code. You may use standard library containers like `std::vector` and `std::initializer_list` to handle dynamic array sizing and inputs. **Remark:** It is advisable that you use GenAI to learn `std::vector` and `std::initializer_list` by guided learning mode or study mode.
3. Write a `main.cpp` file that demonstrates your class solving addition, subtraction, multiplication, and evaluation. Include comments explaining your algorithms (especially for polynomial multiplication and Horner's method evaluation).

Step 3: Comparative Analysis and Writing Report

Compare your custom code against the GenAI baseline. **You must write an extensive report about the differences and analyse the differences.**

Note that your solution is not necessarily better than the GenAI solution in the first round of writing, or even better than it at all. Your task is to analyse the differences and explain what you have learned from it.

5 Understanding `numpy.poly1d` in Python

Before implementing your C++ class, you must understand how NumPy's classic `poly1d` class behaves. In Python, `poly1d` represents a one-dimensional polynomial. It can be initialized with its coefficients, printed in a readable format, evaluated at specific points, and manipulated algebraically.

Here are comprehensive examples demonstrating its behavior:

```

import numpy as np

# 1. Initialization
# We define P(x) = 3x^2 + 2x + 1 by passing coefficients in descending order of power
p = np.poly1d([3, 2, 1])
print("Polynomial P(x):")
print(p)

# 2. Evaluation
# We evaluate P(2) -> 3(2)^2 + 2(2) + 1 = 12 + 4 + 1 = 17
print(f"P(2) = {p(2)}\n")

# 3. Basic Arithmetic
# Let Q(x) = 1x - 5
q = np.poly1d([1, -5])
print("Polynomial Q(x):")
print(q)

print("\nAddition: P(x) + Q(x)")
print(p + q)

print("\nSubtraction: P(x) - Q(x)")
print(p - q)

print("\nMultiplication: P(x) * Q(x)")
print(p * q)

# 4. Roots and Properties
print(f"\nOrder/Degree of P(x): {p.order}")
print(f"Coefficients of P(x): {p.coeffs}")
print(f"Roots of Q(x) (where Q(x) = 0): {q.roots}")

```

Polynomial P(x):

2

3 x + 2 x + 1

P(2) = 17

Polynomial Q(x):

1 x - 5

Addition: P(x) + Q(x)

2

3 x + 3 x - 4

Subtraction: P(x) - Q(x)

2

3 x + 1 x + 6

Multiplication: P(x) * Q(x)

3

2

3 x - 13 x - 9 x - 5

Order/Degree of P(x): 2

Coefficients of $P(x)$: [3 2 1]

Roots of $Q(x)$ (where $Q(x) = 0$): [5.]